

Adaptive LLM-Based Conversational AI - Project Documentation



Table of Contents

1. [Project Overview](#)
2. [System Architecture](#)
3. [Features](#)
4. [Technology Stack](#)
5. [Project Structure](#)
6. [Setup Instructions](#)
7. [How It Works](#)
8. [API Endpoints](#)
9. [Recent Changes & Enhancements](#)
10. [Usage Guide](#)
11. [Troubleshooting](#)



Project Overview

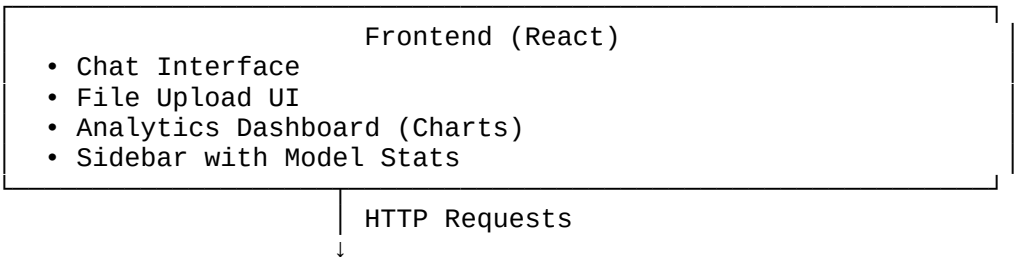
Adaptive LLM-Based Conversational AI is an intelligent chatbot application that automatically adapts its conversational style based on user personality detection. The system uses a local LLM (Large Language Model) running via Ollama and implements RAG (Retrieval-Augmented Generation) to enable context-aware conversations with uploaded documents.

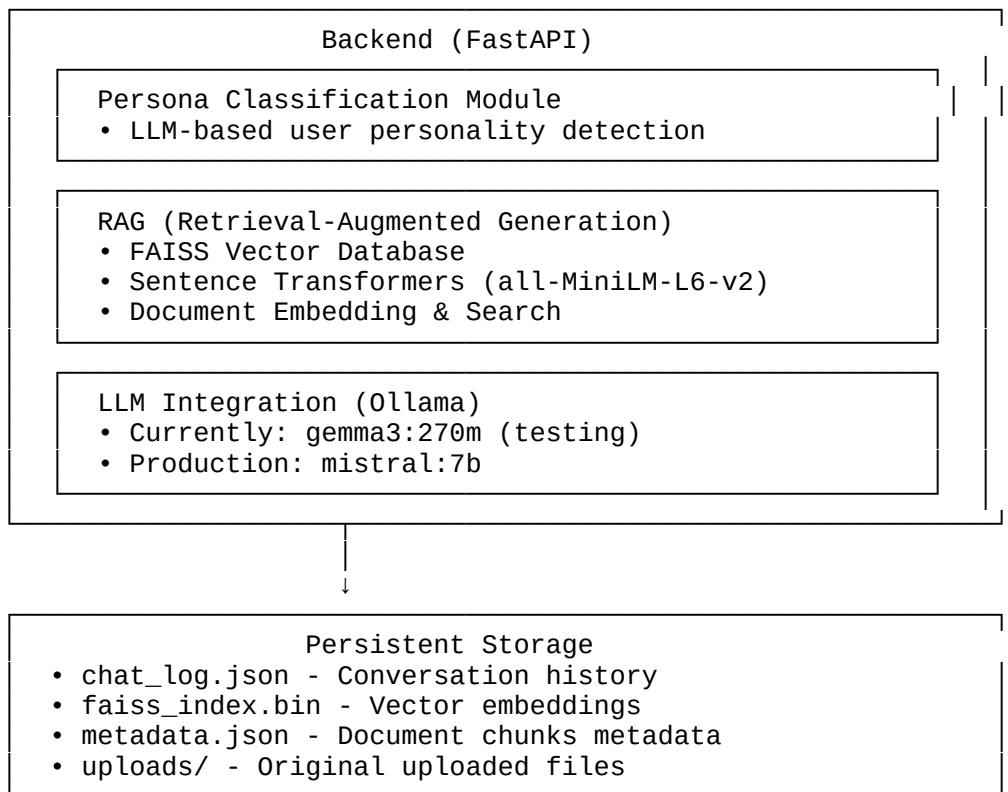
Key Capabilities:

- **Personality Detection:** Automatically classifies users into 3 personas (Verbose, Reserved, Oversharer)
- **Adaptive Responses:** Adjusts tone, length, and style based on detected personality
- **Document Intelligence:** Upload PDFs, TXT, DOC, DOCX files and chat with their content
- **Conversation Memory:** Maintains context across sessions with chat history
- **Analytics Dashboard:** Visual insights into personality distribution and model capabilities
- **Fully Local:** Runs entirely on your machine using Ollama (no cloud dependencies)



System Architecture





✨ Features

1. Intelligent Personality Detection

- Analyzes user messages in real-time
- Classifies into 3 personality types:
 - **Verbose:** Long, detailed messages
 - **Reserved:** Short, minimal responses
 - **Oversharer:** Emotional, personal disclosures
- Adapts response style automatically

2. RAG-Powered Document Chat

- Upload documents (PDF, TXT, DOC, DOCX)
- Automatic text extraction and chunking
- Vector embedding generation using Sentence Transformers
- Semantic search using FAISS
- Context-aware responses based on document content

3. Conversation Management

- Persistent chat history across sessions
- Last 4 messages context window
- Typing indicator for better UX
- Timestamped messages

4. Analytics & Insights

- **Model Capabilities Radar Chart:** Shows context memory, adaptivity, speed, model size, locality
- **Persona Distribution Bar Chart:** Visualizes personality classification statistics
- **Conversation Insights:** Recent chat history preview
- **Document Management:** List of uploaded files with delete option

5. Modern UI/UX

- Responsive sidebar with collapsible sections
- WhatsApp-like chat interface
- Color-coded personas and RAG indicators
- File upload with drag-and-drop support
- Real-time status notifications



Technology Stack

Frontend

- **React 18** - UI framework
- **React Bootstrap** - UI components
- **Recharts** - Data visualization
- **Font Awesome** - Icons

Backend

- **FastAPI** - Web framework
- **Ollama** - LLM integration
- **LangChain Community** - LLM orchestration
- **FAISS** - Vector similarity search
- **Sentence Transformers** - Text embeddings
- **PyPDF2** - PDF processing
- **python-docx** - Word document processing

Machine Learning

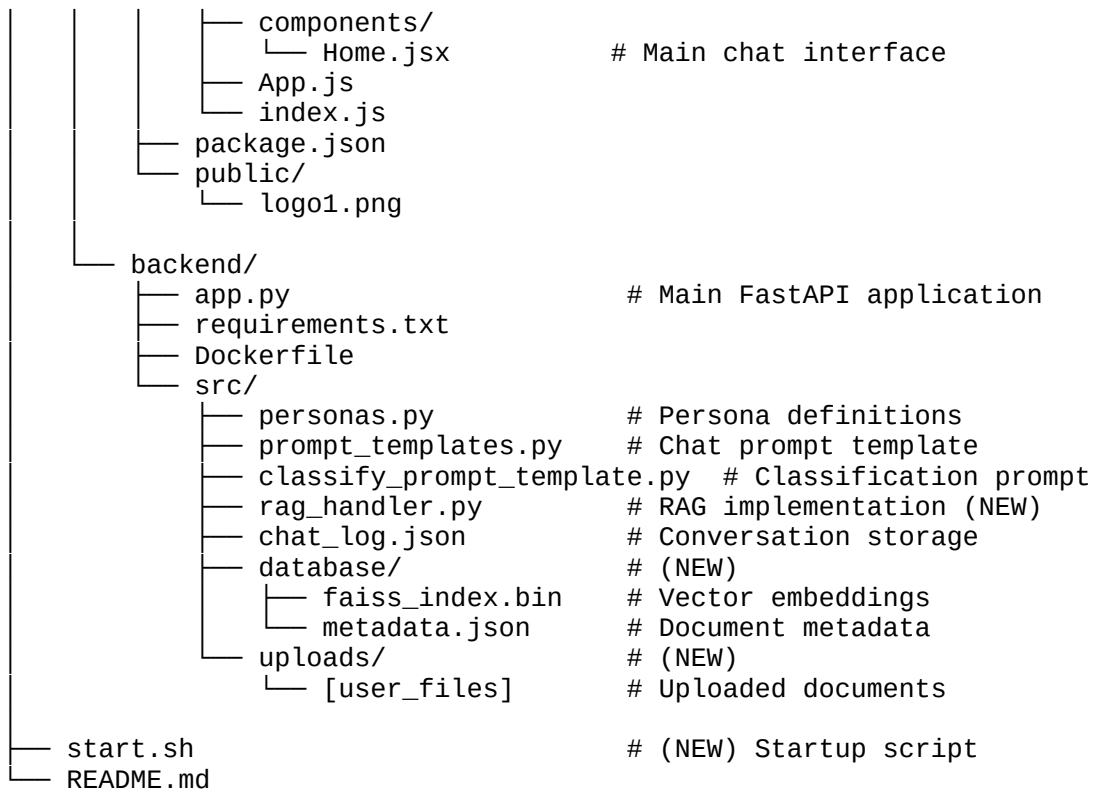
- **LLM:** gemma3:270m (testing) / mistral:7b (production)
- **Embeddings:** all-MiniLM-L6-v2 (384 dimensions)
- **Vector DB:** FAISS IndexFlatL2



Project Structure

Adaptive-LLM-Based-Conversational-AI/

```
|
|— website/
|   |— frontend/
|       |— src/
```



Setup Instructions

Prerequisites

```
# 1. Install Python 3.9+
python --version

# 2. Install Node.js 18+
node --version

# 3. Install Ollama
# Visit: https://ollama.ai/download
ollama --version

# 4. Pull LLM model
ollama pull gemma3:270m # For testing
ollama pull mistral:latest # For production
```

Backend Setup

```
cd website/backend

# Create virtual environment
python -m venv venv
source venv/bin/activate # On Windows: venv\Scripts\activate

# Install dependencies
pip install -r requirements.txt

# Create necessary directories
mkdir -p src/database src/uploads
```

```
# Start backend server
uvicorn app:app --reload
# Backend runs on: http://127.0.0.1:8000
```

Frontend Setup

```
cd website/frontend

# Install dependencies
npm install

# Start development server
npm start
# Frontend runs on: http://localhost:3000
```

One-Command Startup (Recommended)

```
# Install concurrently in frontend
cd website/frontend
npm install --save-dev concurrently

# Run both with one command
npm run dev
```



How It Works

1. User Sends Message

User: "I'm feeling overwhelmed today, I just can't stop thinking about work..."

2. Personality Classification

```
# Backend classifies using LLM
classification_prompt = "Classify as: oversharer, verbose, reserved"
persona = llm.invoke(classification_prompt) # Result: "oversharer"
```

3. RAG Retrieval (if documents uploaded)

```
# Convert user message to embedding
query_embedding = embedder.encode(message)

# Search FAISS for similar document chunks
relevant_docs = rag_handler.search(message, top_k=2)

# Add to context
rag_context = "RELEVANT DOCUMENT CONTEXT:\n" + "\n".join(relevant_docs)
```

4. Response Generation

```
# Build prompt with:
# - Persona description ("supportive tone, 2-3 sentences")
# - Conversation history (last 4 messages)
# - RAG context (relevant document chunks)
# - User message

response = llm.invoke(complete_prompt)
```

5. Frontend Display

Bot: I hear you - work stress can be really tough. 😞 It's okay to feel this way. Have you tried taking short breaks?

[🤖 oversharer] [📄 2 docs]

API Endpoints

Chat Endpoints

POST /chat

Description: Send a message and get AI response

Request Body:

```
{
  "message": "Hello, how are you?"
}
```

Response:

```
{
  "persona_detected": "verbose",
  "response": "Hello! I'm doing well, thank you for asking...",
  "rag_used": true,
  "rag_sources": 2
}
```

GET /chat/history

Description: Retrieve conversation history

Query Parameters:

- user_id (default: "user123")
- limit (default: 10)

Response:

```
{
  "history": [
    {
      "timestamp": "2025-12-26T00:30:15",
      "message": "Hello",
      "response": "Hi there!"
    }
  ]
}
```

Document Management Endpoints

POST /upload-document

Description: Upload a document for RAG

Form Data:

- file: PDF, TXT, DOC, or DOCX file

Response:

```
{
  "status": "success",
  "message": "File 'document.pdf' uploaded and indexed successfully",
  "chunks": 15
}
```

GET /uploaded-files

Description: List all uploaded documents

Response:

```
{
  "files": [
    {
      "name": "document.pdf",
      "uploadedAt": "12:30 PM, Dec 26"
    }
  ],
  "total_chunks": 15
}
```

DELETE /delete-file/{filename}

Description: Delete an uploaded document

Response:

```
{
  "status": "success",
  "message": "File 'document.pdf' deleted"
}
```

Analytics Endpoints

GET /persona-counts

Description: Get personality distribution statistics

Response:

```
{
  "verbose": 45,
  "reserved": 23,
  "oversharer": 12
}
```

GET /rag-stats

Description: Get RAG system statistics

Response:

```
{
  "total_chunks": 150,
}
```

```
"total_documents": 5,  
"embedding_dimension": 384  
}
```



Recent Changes & Enhancements



Phase 1: Persona Display Enhancement

Date: December 2025

Changes Made:

1. **Modified Backend Response** (`app.py`)
 - Added `persona_detected` field to `/chat` endpoint response
 - Returns detected personality with each message
2. **Updated Frontend State** (`Home.jsx`)
 - Added `persona` field to chat message state
 - Stores personality type with each bot message
3. **Enhanced Message Display**
 - Added persona badge inside message card
 - Badge shows: 🤖 `verbose`, 🤖 `reserved`, or 🤖 `oversharer`
 - Styled with purple theme (`#7678ee` background)

Code Location:

- **Backend:** `app.py` line ~95 (return statement in `/chat`)
 - **Frontend:** `Home.jsx` lines ~50 (state update), ~280 (display)
-



Phase 2: RAG Implementation

Date: December 2025

Changes Made:

1. **Created RAG Handler Module** (`src/rag_handler.py`)
 - Implemented `RAGHandler` class
 - Functions:
 - `extract_text_from_file()` - Extracts text from PDF/DOC/TXT
 - `chunk_text()` - Splits text into 500-word chunks
 - `add_document()` - Generates embeddings and stores in FAISS
 - `search()` - Semantic search for relevant chunks
 - `_safe_save()` - Persists FAISS index and metadata
2. **Integrated RAG into Chat Flow** (`app.py`)
 - Import: `from src.rag_handler import rag_handler`

- Added RAG retrieval before LLM call (line ~88)
- Appends relevant document chunks to prompt context
- Returns `rag_used` and `rag_sources` in response

3. Added Document Upload Endpoint (`app.py`)

- `POST /upload-document` - Accepts file upload
- Validates file type (PDF, TXT, DOC, DOCX)
- Saves to `src/uploads/` directory
- Triggers embedding generation

4. Created Persistence Layer

- `src/database/faiss_index.bin` - Vector embeddings
- `src/database/metadata.json` - Document chunks + filenames
- `src/uploads/` - Original uploaded files

Dependencies Added (`requirements.txt`):

```
sentence-transformers
PyPDF2
python-docx
python-multipart
faiss-cpu
```

Code Location:

- **RAG Module:** `src/rag_handler.py` (NEW FILE)
 - **Backend Integration:** `app.py` lines ~10 (import), ~88 (RAG retrieval), ~120 (upload endpoint)
-

Phase 3: Frontend File Upload UI

Date: December 2025

Changes Made:

1. Added File Upload Button (`Home.jsx`)

- Paperclip icon in header (right side)
- Hidden file input with `accept=".pdf, .txt, .doc, .docx"`
- Positioned at `right: 60px, top: 15px`

2. Implemented Upload Handler

- `handleFileUpload()` function
- Creates `FormData` and sends to backend
- Shows upload status notification
- Updates `uploadedFiles` state

3. Added Upload Status Notification

- Appears below header when uploading
- Shows: "Uploading...", " Success", or " Error"

- Auto-dismisses after 5 seconds

4. Created Uploaded Files List (Sidebar)

- New Accordion item: "📄 Uploaded Documents"
- Displays file name and upload time
- Shows document count in header
- Delete button (🗑️) for each file

5. Added RAG Indicator Badges

- Shows 📄 2 docs when RAG context used
- Green badge next to persona badge
- Only appears when document chunks retrieved

6. Context Indicator Below Input

- Shows: "💡 3 documents available for context"
- Appears when files uploaded
- Helps user know RAG is active

Code Location:

- **Upload Button:** Home .jsx lines ~236-250
 - **Upload Handler:** Home .jsx lines ~115-135
 - **File List:** Home .jsx lines ~195-220
 - **RAG Badges:** Home .jsx lines ~280-295
-

✅ Phase 4: Document Persistence

Date: December 2025

Changes Made:

1. Added File List Endpoint (app.py)

- GET /uploaded-files
- Scans src/uploads/ directory
- Returns file names and upload timestamps
- Includes total chunks count

2. Frontend Auto-Load on Refresh (Home.jsx)

- Added useEffect to fetch uploaded files on mount
- Populates uploadedFiles state
- Files persist across page refreshes

3. Delete File Endpoint (app.py)

- DELETE /delete-file/{filename}
- Removes file from disk
- Rebuilds FAISS index from remaining files
- Returns success/error status

4. Frontend Delete Handler (Home.jsx)

- `handleDeleteFile()` function
- Confirms deletion with user
- Updates UI immediately
- Shows status notification

5. Improved Error Handling (`rag_handler.py`)

- Added try-catch blocks in all methods
- Safe save with temporary files
- Corrupted index recovery
- Better logging for debugging

Code Location:

- **List Endpoint:** `app.py` lines ~140-160
 - **Delete Endpoint:** `app.py` lines ~162-185
 - **Frontend Fetch:** `Home.jsx` lines ~75-85
 - **Delete Handler:** `Home.jsx` lines ~137-155
-

Phase 5: One-Command Startup

Date: December 2025

Changes Made:

1. Added Concurrently Package

- Installed in frontend: `npm install --save-dev concurrently`
- Allows running multiple npm scripts simultaneously

2. Updated `package.json` Scripts

```
{
  "backend": "cd ../backend && uvicorn app:app --reload",
  "dev": "concurrently \"npm start\" \"npm run backend\"",
  "dev:all": "concurrently -n \"FRONTEND, BACKEND\" -c \"blue, green\" \"npm start\" \"npm run backend\""
}
```

3. Created Startup Scripts

- `start.sh` for Linux/Mac
- `start.bat` for Windows
- `start.py` for cross-platform Python approach

Usage:

```
cd website/frontend
npm run dev
```

Code Location:

- **Package.json:** `website/frontend/package.json` (scripts section)
- **Shell Script:** `start.sh` (root directory)



Feature Comparison: Before vs After

Feature	Before	After Enhancement
Persona Detection	Hidden from user	✓ Visible badge on each message
Document Upload	✗ Not available	✓ Click-to-upload with status
RAG Integration	✗ No document context	✓ Semantic search with FAISS
File Management	✗ N/A	✓ List, view, delete uploaded files
Persistence	⚠ Lost on refresh	✓ Files/embeddings persist
RAG Visibility	✗ N/A	✓ Badge shows when docs used
Startup	⚠ Two terminals	✓ One command (npm run dev)
Error Handling	⚠ Basic	✓ Comprehensive try-catch



Usage Guide

For End Users

1. Start Chatting

- Type a message in the input box
- Press Enter or click send button (✈️)
- Watch for typing indicator (...)
- Bot responds with adapted style

2. Upload Documents

- Click paperclip icon (📎) in header
- Select PDF, TXT, DOC, or DOCX file
- Wait for "✓ File uploaded and indexed" message
- Document appears in sidebar under "📄 Uploaded Documents"

3. Chat with Documents

- After uploading, ask questions about the content
- Examples:
 - "What is this document about?"
 - "Summarize the key points"
 - "What does it say about [topic]?"
- Look for 📄 2 docs badge to confirm RAG is working

4. View Analytics

- Open sidebar accordion sections:
 - **Model Capabilities:** Radar chart of model stats
 - **Persona Distribution:** Bar chart of personality types
 - **Conversation Insights:** Recent chat history
 - **Uploaded Documents:** List of files with delete option

5. Delete Documents

- Open "📄 Uploaded Documents" in sidebar
- Click 🗑️ next to file name
- Confirm deletion
- File removed from system

For Developers

1. Switch LLM Model

```
# In app.py, change line ~25
llm = Ollama(model="gemma3:270m") # Testing
# TO
llm = Ollama(model="mistral:latest") # Production
```

2. Adjust RAG Parameters

```
# In app.py, line ~88
relevant_docs = rag_handler.search(input.message, top_k=2) # Change top_k

# In rag_handler.py, line ~60
def chunk_text(self, text: str, chunk_size: int = 500): # Change chunk_size
```

3. Modify Persona Definitions

```
# In src/personas.py
PERSONAS = {
    "oversharer": "New description...",
    "verbose": "New description...",
    "reserved": "New description..."
}
```

4. Customize Response Style

```
# In app.py
PERSONA_RESPONSE_STYLE = {
    "oversharer": "Calm, supportive tone. 2-3 sentences.",
    "verbose": "Clear, detailed. 3-4 sentences.",
    "reserved": "Brief, minimal. Max 10 words."
}
```

5. Add New File Types

```
# In rag_handler.py, add to extract_text_from_file()
elif file_type == 'csv':
    import pandas as pd
    df = pd.read_csv(file_path)
    text = df.to_string()
```



Troubleshooting

Issue 1: "FAISS std::system_error"

Solution:

```
# Delete corrupted index
rm -rf src/database/faiss_index.bin src/database/metadata.json

# Restart backend
uvicorn app:app --reload
```

Issue 2: Upload button not visible

Solution:

```
<!-- Add to index.html -->
<link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-
awesome/6.4.0/css/all.min.css" />
```

Issue 3: Backend not finding Ollama

Solution:

```
# Check Ollama is running
ollama list

# Start Ollama service
ollama serve

# Pull model
ollama pull gemma3:270m
```

Issue 4: Frontend can't connect to backend

Solution:

```
// Check CORS settings in app.py
allow_origins=["*"] // Should allow all origins

// Verify backend is running on port 8000
// Check browser console for errors
```

Issue 5: RAG not working after upload

Solution:

```
# Check RAG stats endpoint
# Visit: http://127.0.0.1:8000/rag-stats

# Should show:
{
  "total_chunks": > 0,
  "total_documents": > 0
}

# If 0, check backend logs for errors
```

Issue 6: Files not persisting after restart

Solution:

```
# Ensure directories exist
mkdir -p src/database src/uploads

# Check file permissions
```

```
chmod -R 755 src/database src/uploads
```

```
# Verify files exist  
ls -la src/database/  
ls -la src/uploads/
```